01 LC 46 | Permutations (Medium)
Problem: Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.
Constraints: 1 <= nums.length <= 6; -10 <= nums[i] <= 10; All the integers of nums are unique.

```
class Solution {
public:
  vector<vector<int>> permute(vector<int>& nums) {
    int n = nums.size();
    vector<vector<int>> ans;
    vector<int> t(n);
    vector<bool> vis(n);
    auto dfs = [&](auto&& dfs, int i) -> void {
      if (i == n) {
        ans.emplace_back(t);
        return;
      }
      for (int j = 0; j < n; ++j) {
        if (!vis[j]) {
          vis[j] = true;
          t[i] = nums[j];
          dfs(dfs, i + 1);
          vis[j] = false;
        }
      }
    };
    dfs(dfs, 0);
    return ans;
  }
};
```

22 LC 1415 | The k-th Lexicographical String of All Happy Strings of Length n (Medium)
Problem: Return the kth lexicographically smallest length-n string over a,b,c with no equal neighboring characters. k is one-based; return an empty string if fewer than k valid strings exist.
Constraints: 1 <= n <= 10; 1 <= k <= 100

```
class Solution {
public:
  string getHappyString(int n, int k) {
    if (k > 3 * (1 << (n - 1))) {
      return "";
    }
    string cs = "abc";
    string ans;
    for (int i = 0; i < n; ++i) {
      int remain = 1 << (n - i - 1);
      for (char c : cs) {
        if (!ans.empty() && ans.back() == c) {
          continue;
        }
        if (k <= remain) {
          ans.push_back(c);
          break;
        }
        k -= remain;
      }
    }
    return ans;
  }
};
```

02 LC 78 | Subsets (Medium)
Problem: Given an integer array nums of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets. Return the solution in any order.
Constraints: 1 <= nums.length <= 10; -10 <= nums[i] <= 10; All the numbers of nums are unique.

```
class Solution {
public:
  vector<vector<int>> subsets(vector<int>& nums) {
    vector<vector<int>> ans;
    vector<int> t;
    function<void(int)> dfs = [&](int i) -> void {
      if (i == nums.size()) {
        ans.push_back(t);
        return;
      }
      dfs(i + 1);
      t.push_back(nums[i]);
      dfs(i + 1);
      t.pop_back();
    };
    dfs(0);
    return ans;
  }
};
```

39 LC 980 | Unique Paths III (Hard)
Problem: The grid contains one start (1), one end (2), empty cells (0), and obstacles (-1). Count horizontal/vertical paths from start to end that visit every non-obstacle cell exactly once.
Constraints: m == grid.length; n == grid[i].length; 1 <= m, n <= 20; 1 <= m * n <= 20; -1 <= grid[i][j] <= 2; There is exactly one starting cell and one ending cell.

```
class Solution {
public:
  int uniquePathsIII(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    int cnt = 0;
    for (auto& row : grid) {
      for (auto& x : row) {
        cnt += x == 0;
      }
    }
    int dirs[5] = {-1, 0, 1, 0, -1};
    vector<vector<char>> vis =
      vector<vector<char>>(m, vector<char>(n, 0));
    function<int(int, int, int)> dfs = [&](int i,
      int j, int k) -> int {
      if (grid[i][j] == 2) {
        return k == cnt + 1 ? 1 : 0;
      }
      int ans = 0;
      for (int h = 0; h < 4; ++h) {
        int x = i + dirs[h], y = j + dirs[h + 1];
        if (x >= 0 && x < m && y >= 0 && y < n &&
          !vis[x][y] && grid[x][y] != -1) {
          vis[x][y] = true;
          ans += dfs(x, y, k + 1);
          vis[x][y] = false;
        }
      }
      return ans;
    };
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        if (grid[i][j] == 1) {
          vis[i][j] = true;
          return dfs(i, j, 0);
        }
      }
    }
    return 0;
  }
};
```

03 LC 39 | Combination Sum (Medium)

Problem: Return all distinct combinations of candidates summing to target. Each distinct positive candidate may be reused without limit; combinations differing only in order are identical.
Constraints: 1 <= candidates.length <= 30; 2 <= candidates[i] <= 40; All elements of candidates are distinct.; 1 <= target <= 40

```
class Solution {
public:
  vector<vector<int>> combinationSum(vector<int>&
    candidates, int target) {
    ranges::sort(candidates);
    vector<vector<int>> ans;
    vector<int> t;
    function<void(int, int)> dfs = [&](int i, int
      s) {
      if (s == 0) {
        ans.emplace_back(t);
        return;
      }
      if (s < candidates[i]) {
        return;
      }
      for (int j = i; j < candidates.size(); ++j)
        {
        t.push_back(candidates[j]);
        dfs(j, s - candidates[j]);
        t.pop_back();
      }
    };
    dfs(0, target);
    return ans;
  }
};
```

11 LC 40 | Combination Sum II (Medium)

Problem: Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination. Note: The solution set must not contain duplicate combinations.
Constraints: 1 <= candidates.length <= 100; 1 <= candidates[i] <= 50; 1 <= target <= 30

```
class Solution {
public:
  vector<vector<int>> combinationSum2(vector<int>&
    candidates, int target) {
    ranges::sort(candidates);
    vector<vector<int>> ans;
    vector<int> t;
    function<void(int, int)> dfs = [&](int i, int
      s) {
      if (s == 0) {
        ans.emplace_back(t);
        return;
      }
      if (i >= candidates.size() || s <
        candidates[i]) {
        return;
      }
      for (int j = i; j < candidates.size(); ++j)
        {
        if (j > i && candidates[j] == candidates[j
          - 1]) {
          continue;
        }
        t.emplace_back(candidates[j]);
        dfs(j + 1, s - candidates[j]);
        t.pop_back();
      }
    };
    dfs(0, target);
    return ans;
  }
};
```

04 LC 77 | Combinations (Medium)

Problem: Given two integers n and k, return all possible combinations of k numbers chosen from the range [1, n]. You may return the answer in any order.
Constraints: 1 <= n <= 20; 1 <= k <= n

```
class Solution {
public:
  vector<vector<int>> combine(int n, int k) {
    vector<vector<int>> ans;
    vector<int> t;
    function<void(int)> dfs = [&](int i) {
      if (t.size() == k) {
        ans.emplace_back(t);
        return;
      }
      if (i > n) {
        return;
      }
      t.emplace_back(i);
      dfs(i + 1);
      t.pop_back();
      dfs(i + 1);
    };
    dfs(1);
    return ans;
  }
};
```

06 LC 79 | Word Search (Medium)

Problem: Given an m x n grid of characters board and a string word, return true if word exists in the grid. The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.
Constraints: m == board.length; n = board[i].length; 1 <= m, n <= 6; 1 <= word.length <= 15; board and word consists of only lowercase and uppercase English letters.

```
class Solution {
public:
  bool exist(vector<vector<char>>& board, string
    word) {
    int m = board.size(), n = board[0].size();
    int dirs[5] = {-1, 0, 1, 0, -1};
    function<bool(int, int, int)> dfs = [&](int i,
      int j, int k) -> bool {
      if (k == word.size() - 1) {
        return board[i][j] == word[k];
      }
      if (board[i][j] != word[k]) {
        return false;
      }
      char c = board[i][j];
      board[i][j] = '0';
      for (int u = 0; u < 4; ++u) {
        int x = i + dirs[u], y = j + dirs[u + 1];
        if (x >= 0 && x < m && y >= 0 && y < n &&
          board[x][y] != '0' && dfs(x, y, k + 1))
          {
          return true;
        }
      }
      board[i][j] = c;
      return false;
    };
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        if (dfs(i, j, 0)) {
          return true;
        }
      }
    }
    return false;
  }
};
```

05 LC 22 | Generate Parentheses (Medium)

Problem: Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.
Constraints: 1 <= n <= 8

```
class Solution {
public:
  vector<string> generateParenthesis(int n) {
    vector<string> ans;
    auto dfs = [&](auto&& dfs, int l, int r,
      string t) -> void {
      if (l > n || r > n || l < r) {
        return;
      }
      if (l == n && r == n) {
        ans.push_back(t);
        return;
      }
      dfs(dfs, l + 1, r, t + "(");
      dfs(dfs, l, r + 1, t + ")");
    };
    dfs(dfs, 0, 0, "");
    return ans;
  }
};
```

24 LC 1863 | Sum of All Subset XOR Totals (Easy)

Problem: Return the sum of the XORs of every subset of nums. Each index is independently selected, so equal values at different indices count as separate choices. The empty subset contributes 0.
Constraints: 1 <= nums.length <= 12; 1 <= nums[i] <= 20

```
class Solution {
public:
  int subsetXORSum(vector<int>& nums) {
    unsigned bits = 0;
    for (int x : nums) bits |=
      static_cast<unsigned>(x);
    return static_cast<int>(bits << (nums.size() -
      1));
  }
};
```

25 LC 2044 l Count Number of Maximum Bitwise-OR Subsets (Medium)

Problem: Count nonempty index subsets whose bitwise OR is the maximum possible OR of any subset of nums. Equal values at different indices are separate choices.
Constraints: 1 <= nums.length <= 16; 1 <= nums[i] <= 10^5

```
class Solution {
public:
  int countMaxOrSubsets(vector<int>& nums) {
    int ans = 0;
    int mx = accumulate(nums.begin(), nums.end(),
      0, bit_or<int>());
    auto dfs = [&](auto&& dfs, int i, int t) {
      if (i == nums.size()) {
        if (t == mx) {
          ans++;
        }
        return;
      }
      dfs(dfs, i + 1, t);
      dfs(dfs, i + 1, t | nums[i]);
    };
    dfs(dfs, 0, 0);
    return ans;
  }
};
```

07 LC 51 | N-Queens (Hard)

Problem: The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other. Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order. Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.
Constraints: 1 <= n <= 9

```
class Solution {
public:
  vector<vector<string>> solveNQueens(int n) {
    vector<int> col(n);
    vector<int> dg(n << 1);
    vector<int> udg(n << 1);
    vector<vector<string>> ans;
    vector<string> t(n, string(n, '.'));
    function<void(int)> dfs = [&](int i) -> void {
      if (i == n) {
        ans.push_back(t);
        return;
      }
      for (int j = 0; j < n; ++j) {
        if (col[j] + dg[i + j] + udg[n - i + j] ==
          0) {
          t[i][j] = 'Q';
          col[j] = dg[i + j] = udg[n - i + j] = 1;
          dfs(i + 1);
          col[j] = dg[i + j] = udg[n - i + j] = 0;
          t[i][j] = '.';
        }
      }
    };
    dfs(0);
    return ans;
  }
};
```

14 LC 131 | Palindrome Partitioning (Medium)

Problem: Given a string s, partition s such that every substring of the partition is a palindrome. Return all possible palindrome partitioning of s.
Constraints: 1 <= s.length <= 16; s contains only lowercase English letters.

```
class Solution {
public:
  vector<vector<string>> partition(string s) {
    int n = s.size();
    vector<vector<char>> f =
      vector<vector<char>>(n, vector<char>(n, 1));
    for (int i = n - 1; i >= 0; --i) {
      for (int j = i + 1; j < n; ++j) {
        f[i][j] = s[i] == s[j] && f[i + 1][j - 1];
      }
    }
    vector<vector<string>> ans;
    vector<string> t;
    auto dfs = [&](auto&& dfs, int i) -> void {
      if (i == n) {
        ans.emplace_back(t);
        return;
      }
      for (int j = i; j < n; ++j) {
        if (f[i][j]) {
          t.emplace_back(s.substr(i, j - i + 1));
          dfs(dfs, j + 1);
          t.pop_back();
        }
      }
    };
    dfs(dfs, 0);
    return ans;
  }
};
```

08 LC 37 | Sudoku Solver (Hard)

Problem: Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: - Each of the digits 1-9 must occur exactly once in each row. - Each of the digits 1-9 must occur exactly once in each column. - Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells.

Constraints: board.length == 9; board[i].length == 9; board[i][j] is a digit or '.'.; It is guaranteed that the input board has only one solution.

```
class Solution {
public:
  void solveSudoku(vector<vector<char>>& board) {
    bool row[9][9] = {false};
    bool col[9][9] = {false};
    bool block[3][3][9] = {false};
    bool ok = false;
    vector<pair<int, int>> t;
    for (int i = 0; i < 9; ++i) {
      for (int j = 0; j < 9; ++j) {
        if (board[i][j] == '.') {
          t.push_back({i, j});
        } else {
          int v = board[i][j] - '1';
          row[i][v] = col[j][v] = block[i / 3][j /
            3][v] = true;
        }
      }
    }
    auto dfs = [&](auto&& dfs, int k) -> void {
      if (k == t.size()) {
        ok = true;
        return;
      }
      int i = t[k].first, j = t[k].second;
      for (int v = 0; v < 9; ++v) {
        if (!row[i][v] && !col[j][v] && !block[i /
          3][j / 3][v]) {
          row[i][v] = col[j][v] = block[i / 3][j /
            3][v] = true;
          board[i][j] = v + '1';
          dfs(dfs, k + 1);
          row[i][v] = col[j][v] = block[i / 3][j /
            3][v] = false;
        }
        if (ok) {
          return;
        }
      }
    };
    dfs(dfs, 0);
  }
};
```

49 LC 89 | Gray Code (Medium)

Problem: Return a length-2^n Gray-code sequence beginning with 0. Every value in [0,2^n-1] occurs once, and adjacent values, including the final/first pair, differ in exactly one bit.

Constraints: 1 <= n <= 16

```
class Solution {
public:
  vector<int> grayCode(int n) {
    vector<int> ans;
    for (int i = 0; i < 1 << n; ++i) {
      ans.push_back(i ^ (i >> 1));
    }
    return ans;
  }
};
```

09 LC 90 | Subsets II (Medium)

Problem: Given an integer array nums that may contain duplicates, return all possible subsets (the power set). The solution set must not contain duplicate subsets. Return the solution in any order.

Constraints: 1 <= nums.length <= 10; -10 <= nums[i] <= 10

```
class Solution {
public:
  vector<vector<int>> subsetsWithDup(vector<int>&
    nums) {
    ranges::sort(nums);
    vector<vector<int>> ans;
    vector<int> t;
    int n = nums.size();
    auto dfs = [&](auto&& dfs, int i) {
      if (i >= n) {
        ans.push_back(t);
        return;
      }
      t.push_back(nums[i]);
      dfs(dfs, i + 1);
      t.pop_back();
      while (i + 1 < n && nums[i + 1] == nums[i])
        {
        ++i;
      }
      dfs(dfs, i + 1);
    };
    dfs(dfs, 0);
    return ans;
  }
};
```

37 LC 842 | Split Array into Fibonacci Sequence (Medium)

Problem: Split num into at least three nonnegative signed 32-bit integers, each after the first two being their sum. Use all digits in order; forbid leading zeros except for zero itself. Return any valid sequence, or an empty array.

Constraints: 1 <= num.length <= 200; num contains only digits.

```
class Solution {
public:
  vector<int> splitIntoFibonacci(string num) {
    int n = num.size();
    vector<int> ans;
    function<bool(int)> dfs = [&](int i) -> bool {
      if (i == n) {
        return ans.size() > 2;
      }
      long long x = 0;
      for (int j = i; j < n; ++j) {
        if (j > i && num[i] == '0') {
          break;
        }
        x = x * 10 + num[j] - '0';
        if (x > INT_MAX || (ans.size() > 1 && x >
          (long long) ans[ans.size() - 1] +
          ans[ans.size() - 2])) {
          break;
        }
        if (ans.size() < 2 || x == (long long)
          ans[ans.size() - 1] + ans[ans.size() -
          2]) {
          ans.push_back(x);
          if (dfs(j + 1)) {
            return true;
          }
          ans.pop_back();
        }
      }
      return false;
    };
    dfs(0);
    return ans;
  }
};
```

10 LC 47 | Permutations II (Medium)

Problem: Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order.
Constraints: 1 <= nums.length <= 8; -10 <= nums[i] <= 10

```
class Solution {
public:
  vector<vector<int>> permuteUnique(vector<int>&
    nums) {
    ranges::sort(nums);
    int n = nums.size();
    vector<vector<int>> ans;
    vector<int> t(n);
    vector<bool> vis(n);
    auto dfs = [&](auto&& dfs, int i) {
      if (i == n) {
        ans.emplace_back(t);
        return;
      }
      for (int j = 0; j < n; ++j) {
        if (vis[j] || (j && nums[j] == nums[j - 1]
          && !vis[j - 1])) {
          continue;
        }
        t[i] = nums[j];
        vis[j] = true;
        dfs(dfs, i + 1);
        vis[j] = false;
      }
    };
    dfs(dfs, 0);
    return ans;
  }
};
```

21 LC 1239 | Maximum Length of a Concatenated String with Unique Characters (Medium)

Problem: You are given an array of strings arr. A string s is formed by the concatenation of a subsequence of arr that has unique characters. Return the maximum possible length of s. A subsequence is an array that can be derived from another array by deleting some or no elements without changing the order of the remaining elements.
Constraints: 1 <= arr.length <= 16; 1 <= arr[i].length <= 26; arr[i] contains only lowercase English letters.

```
class Solution {
public:
  int maxLength(vector<string>& arr) {
    vector<int> s = {0};
    int ans = 0;
    for (const string& t : arr) {
      int x = 0;
      for (char c : t) {
        int b = c - 'a';
        if ((x >> b & 1) == 1) {
          x = 0;
          break;
        }
        x |= 1 << b;
      }
      if (x > 0) {
        for (int i = s.size() - 1; i >= 0; --i) {
          int y = s[i];
          if ((x & y) == 0) {
            s.push_back(x | y);
            ans = max(ans,
              popcount(static_cast<unsigned int>(x
              | y)));
          }
        }
      }
    }
    return ans;
  }
};
```

12 LC 216 | Combination Sum III (Medium)

Problem: Find all valid combinations of k numbers that sum up to n such that the following conditions are true: - Only numbers 1 through 9 are used. - Each number is used at most once. Return a list of all possible valid combinations. The list must not contain the same combination twice, and the combinations may be returned in any order.
Constraints: 2 <= k <= 9; 1 <= n <= 60

```
class Solution {
public:
  vector<vector<int>> combinationSum3(int k, int
    n) {
    vector<vector<int>> ans;
    vector<int> t;
    function<void(int, int)> dfs = [&](int i, int
      s) {
      if (s == 0) {
        if (t.size() == k) {
          ans.emplace_back(t);
        }
        return;
      }
      if (i > 9 || i > s || t.size() >= k) {
        return;
      }
      t.emplace_back(i);
      dfs(i + 1, s - i);
      t.pop_back();
      dfs(i + 1, s);
    };
    dfs(1, n);
    return ans;
  }
};
```

17 LC 698 | Partition to K Equal Sum Subsets (Medium)

Problem: Given an integer array nums and an integer k, return true if it is possible to divide this array into k non-empty subsets whose sums are all equal.
Constraints: 1 <= k <= nums.length <= 16; 1 <= nums[i] <= 10^4; The frequency of each element is in the range [1, 4].

```
class Solution {
public:
  bool canPartitionKSubsets(vector<int>& nums, int
    k) {
    int s = accumulate(nums.begin(), nums.end(),
      0);
    if (s % k) {
      return false;
    }
    s /= k;
    ranges::sort(nums);
    int n = nums.size();
    vector<char> f = vector<char>(1 << n, 0);
    vector<int> cur = vector<int>(1 << n, 0);
    f[0] = 1;
    for (int i = 0; i < 1 << n; ++i) {
      if (!f[i]) {
        continue;
      }
      for (int j = 0; j < n; ++j) {
        if (cur[i] + nums[j] > s) {
          break;
        }
        if ((i >> j & 1) == 0) {
          f[i | 1 << j] = true;
          cur[i | 1 << j] = (cur[i] + nums[j]) %
            s;
        }
      }
    }
    return f[(1 << n) - 1];
  }
};
```

13 LC 17 | Letter Combinations of a Phone Number (Medium)

Problem: Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order. A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.
Constraints: 1 <= digits.length <= 4; digits[i] is a digit in the range ['2', '9'].

```
class Solution {
public:
  vector<string> letterCombinations(string digits)
    {
    if (digits.empty()) {
      return {};
    }
    vector<string> d = {"abc", "def", "ghi",
      "jkl", "mno", "pqrs", "tuv", "wxyz"};
    vector<string> ans;
    string t;
    function<void(int)> dfs = [&](int i) {
      if (i >= digits.size()) {
        ans.push_back(t);
        return;
      }
      for (auto& c : d[digits[i] - '2']) {
        t.push_back(c);
        dfs(i + 1);
        t.pop_back();
      }
    };
    dfs(0);
    return ans;
  }
};
```

28 LC 2698 | Find the Punishment Number of an Integer (Medium)

Problem: Given a positive integer n, return the punishment number of n. The punishment number of n is defined as the sum of the squares of all integers i such that: - 1 <= i <= n - The decimal representation of i * i can be partitioned into contiguous substrings such that the sum of the integer values of these substrings equals i.
Constraints: 1 <= n <= 1000

```
class Solution {
public:
  int punishmentNumber(int n) {
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
      int x = i * i;
      string s = to_string(x);
      if (check(s, 0, i)) {
        ans += x;
      }
    }
    return ans;
  }
  bool check(const string& s, int i, int x) {
    int m = s.size();
    if (i >= m) {
      return x == 0;
    }
    int y = 0;
    for (int j = i; j < m; ++j) {
      y = y * 10 + s[j] - '0';
      if (y > x) {
        break;
      }
      if (check(s, j + 1, x - y)) {
        return true;
      }
    }
    return false;
  }
};
```

15 LC 93 | Restore IP Addresses (Medium)

Problem: Insert exactly three dots into digit string s to form every valid IPv4 address. Each of its four parts must be 0..255 with no leading zero unless the part is '0'. Do not reorder or discard digits.
Constraints: 1 <= s.length <= 20; s consists of digits only.

```
class Solution {
public:
  vector<string> restoreIpAddresses(string s) {
    int n = s.size();
    vector<string> ans;
    vector<string> t;
    function<void(int)> dfs = [&](int i) {
      if (i >= n && t.size() == 4) {
        ans.push_back(t[0] + "." + t[1] + "." +
          t[2] + "." + t[3]);
        return;
      }
      if (i >= n || t.size() >= 4) {
        return;
      }
      int x = 0;
      for (int j = i; j < min(n, i + 3); ++j) {
        x = x * 10 + s[j] - '0';
        if (x > 255 || (j > i && s[i] == '0')) {
          break;
        }
        t.push_back(s.substr(i, j - i + 1));
        dfs(j + 1);
        t.pop_back();
      }
    };
    dfs(0);
    return ans;
  }
};
```

32 LC 140 | Word Break II (Hard)

Problem: Given a string s and a dictionary of strings wordDict, add spaces in s to construct a sentence where each word is a valid dictionary word. Return all such possible sentences in any order. Note that the same word in the dictionary may be reused multiple times in the segmentation.
Constraints: 1 <= s.length <= 20; 1 <= wordDict.length <= 1000; 1 <= wordDict[i].length <= 10; s and wordDict[i] consist of only lowercase English letters.; All the strings of wordDict are unique.; Input is generated in a way that the length of the answer doesn't exceed 10^5.

```
class Solution {
public:
  vector<string> wordBreak(string s,
    vector<string>& wordDict) {
    unordered_set<string> words(wordDict.begin(),
      wordDict.end());
    int n = s.size();
    vector<optional<vector<string>>> memo(n + 1);
    memo[n] = vector<string>{""};
    auto dfs = [&](auto&& self, int start) ->
      const vector<string>& {
      if (memo[start]) return *memo[start];
      vector<string> result;
      for (int end = start + 1; end <= n; ++end) {
        string word = s.substr(start, end -
          start);
        if (!words.contains(word)) continue;
        for (const string& tail : self(self, end))
          result.push_back(word + (tail.empty() ?
            "" : " " + tail));
      }
      memo[start] = move(result);
      return *memo[start];
    };
    return dfs(dfs, 0);
  }
};
```

16 LC 784 | Letter Case Permutation (Medium)

Problem: Given a string s, you can transform every letter individually to be lowercase or uppercase to create another string. Return a list of all possible strings we could create. Return the output in any order.

Constraints: 1 <= s.length <= 12; s consists of lowercase English letters, uppercase English letters, and digits.

```
class Solution {
public:
  vector<string> letterCasePermutation(string s) {
    string t = s;
    vector<string> ans;
    auto dfs = [&](auto&& dfs, int i) -> void {
      if (i >= t.size()) {
        ans.push_back(t);
        return;
      }
      dfs(dfs, i + 1);
      if (isalpha(t[i])) {
        t[i] ^= 32;
        dfs(dfs, i + 1);
      }
    };
    dfs(dfs, 0);
    return ans;
  }
};
```

50 LC 95 | Unique Binary Search Trees II (Medium)

Problem: Given an integer n, return all the structurally unique BST's (binary search trees), which has exactly n nodes of unique values from 1 to n. Return the answer in any order.

Constraints: 1 <= n <= 8

```
class Solution {
public:
  vector<TreeNode*> generateTrees(int n) {
    function<vector<TreeNode*>(int, int)> dfs =
      [&](int i, int j) {
      if (i > j) {
        return vector<TreeNode*>{nullptr};
      }
      vector<TreeNode*> ans;
      for (int v = i; v <= j; ++v) {
        auto left = dfs(i, v - 1);
        auto right = dfs(v + 1, j);
        for (auto l : left) {
          for (auto r : right) {
            ans.push_back(new TreeNode(v, l, r));
          }
        }
      }
      return ans;
    };
    return dfs(1, n);
  }
};
```

18 LC 473 | Matchsticks to Square (Medium)

Problem: You are given an integer array matchsticks where matchsticks[i] is the length of the ith matchstick. You want to use all the matchsticks to make one square. You should not break any stick, but you can link them up, and each matchstick must be used exactly one time. Return true if you can make this square and false otherwise.

Constraints: 1 <= matchsticks.length <= 15; 1 <= matchsticks[i] <= 10^8

```
class Solution {
public:
  bool makesquare(vector<int>& matchsticks) {
    int s = 0, mx = 0;
    for (int& v : matchsticks) {
      s += v;
      mx = max(mx, v);
    }
    int x = s / 4, mod = s % 4;
    if (mod != 0 || x < mx) return false;
    ranges::sort(matchsticks, greater<int>());
    vector<int> edges(4);
    return dfs(0, x, matchsticks, edges);
  }
  bool dfs(int u, int x, vector<int>& matchsticks,
    vector<int>& edges) {
    if (u == matchsticks.size()) return true;
    for (int i = 0; i < 4; ++i) {
      if (i > 0 && edges[i - 1] == edges[i])
        continue;
      edges[i] += matchsticks[u];
      if (edges[i] <= x && dfs(u + 1, x,
        matchsticks, edges)) return true;
      edges[i] -= matchsticks[u];
    }
    return false;
  }
};
```

26 LC 2305 | Fair Distribution of Cookies (Medium)

Problem: Distribute whole cookie bags among k children. A child's total is the sum of assigned bags; unfairness is the largest child's total. Return the minimum possible unfairness.

Constraints: 2 <= cookies.length <= 8; 1 <= cookies[i] <= 10^5; 2 <= k <= cookies.length

```
class Solution {
public:
  int distributeCookies(vector<int>& cookies, int
    k) {
    sort(cookies.rbegin(), cookies.rend());
    vector<int> cnt = vector<int>(k, 0);
    int n = cookies.size();
    int ans = 1 << 30;
    function<void(int)> dfs = [&](int i) {
      if (i >= n) {
        ans = *max_element(cnt.begin(),
          cnt.end());
        return;
      }
      for (int j = 0; j < k; ++j) {
        if (cnt[j] + cookies[i] >= ans || (j &&
          cnt[j] == cnt[j - 1])) {
          continue;
        }
        cnt[j] += cookies[i];
        dfs(i + 1);
        cnt[j] -= cookies[i];
      }
    };
    dfs(0);
    return ans;
  }
};
```

19 LC 526 | Beautiful Arrangement (Medium)

Problem: Suppose you have n integers labeled 1 through n. A permutation of those n integers perm (1-indexed) is considered a beautiful arrangement if for every i (1 <= i <= n), either of the following is true: - perm[i] is divisible by i. - i is divisible by perm[i]. Given an integer n, return the number of the beautiful arrangements that you can construct.
Constraints: 1 <= n <= 15

```
class Solution {
public:
  int countArrangement(int n) {
    vector<int> dp(1 << n);
    dp[0] = 1;
    for (unsigned mask = 0; mask < dp.size();
      ++mask) {
      int position = popcount(mask) + 1;
      for (int value = 1; value <= n; ++value)
        if (!(mask & (1U << (value - 1))) &&
          (value % position == 0 || position %
          value == 0))
          dp[mask | (1U << (value - 1))] +=
            dp[mask];
    }
    return dp.back();
  }
};
```

45 LC 2375 | Construct Smallest Number From DI String (Medium)

Problem: You are given a 0-indexed string pattern of length n consisting of the characters 'I' meaning increasing and 'D' meaning decreasing. A 0-indexed string num of length n + 1 is created using the following conditions: - num consists of the digits '1' to '9', where each digit is used at most once. - If pattern[i] == 'I', then num[i] num[i + 1]. Return the lexicographically smallest possible string num that meets the conditions.
Constraints: 1 <= pattern.length <= 8; pattern consists of only the letters 'I' and 'D'.

```
class Solution {
public:
  string smallestNumber(string pattern) {
    string result, pending;
    for (int i = 0; i <=
      static_cast<int>(pattern.size()); ++i) {
      pending += static_cast<char>('1' + i);
      if (i == static_cast<int>(pattern.size()) ||
        pattern[i] == 'I') {
        while (!pending.empty()) { result +=
          pending.back(); pending.pop_back(); }
      }
    }
    return result;
  }
};
```

20 LC 1079 | Letter Tile Possibilities (Medium)

Problem: You have n tiles, where each tile has one letter tiles[i] printed on it. Return the number of possible non-empty sequences of letters you can make using the letters printed on those tiles.
Constraints: 1 <= tiles.length <= 7; tiles consists of uppercase English letters.

```
class Solution {
public:
  int numTilePossibilities(string tiles) {
    int cnt[26]{};
    for (char c : tiles) {
      ++cnt[c - 'A'];
    }
    function<int(int* cnt)> dfs = [&](int* cnt) ->
      int {
      int res = 0;
      for (int i = 0; i < 26; ++i) {
        if (cnt[i] > 0) {
          ++res;
          --cnt[i];
          res += dfs(cnt);
          ++cnt[i];
        }
      }
      return res;
    };
    return dfs(cnt);
  }
};
```

41 LC 1255 | Maximum Score Words Formed by Letters (Hard)

Problem: Choose a subset of words, using each input word at most once, so their combined letters do not exceed the available letter multiplicities. Return the maximum sum of the supplied character scores.
Constraints: 1 <= words.length <= 14; 1 <= words[i].length <= 15; 1 <= letters.length <= 100; letters[i].length == 1; score.length == 26; 0 <= score[i] <= 10; words[i], letters[i] contains only lower case English letters.

```
class Solution {
public:
  int maxScoreWords(vector<string>& words,
    vector<char>& letters, vector<int>& score) {
    int cnt[26]{};
    for (char& c : letters) {
      cnt[c - 'a']++;
    }
    int n = words.size();
    int ans = 0;
    for (int i = 0; i < 1 << n; ++i) {
      int cur[26]{};
      for (int j = 0; j < n; ++j) {
        if (i >> j & 1) {
          for (char& c : words[j]) {
            cur[c - 'a']++;
          }
        }
      }
      bool ok = true;
      int t = 0;
      for (int j = 0; j < 26; ++j) {
        if (cur[j] > cnt[j]) {
          ok = false;
          break;
        }
        t += cur[j] * score[j];
      }
      if (ok && ans < t) {
        ans = t;
      }
    }
    return ans;
  }
};
```

23 LC 1593 | Split a String Into the Max Number of Unique Substrings (Medium)

Problem: Given a string s, return the maximum number of unique substrings that the given string can be split into. You can split string s into any list of non-empty substrings, where the concatenation of the substrings forms the original string. However, you must split the substrings such that all of them are unique. A substring is a contiguous sequence of characters within a string.
Constraints: 1 <= s.length <= 16; s contains only lower case English letters.

```
class Solution {
public:
  int maxUniqueSplit(string s) {
    unordered_set<string> st;
    int n = s.size();
    int ans = 0;
    auto dfs = [&](auto&& dfs, int i) -> void {
      if (st.size() + n - i <= ans) {
        return;
      }
      if (i >= n) {
        ans = max(ans, (int) st.size());
        return;
      }
      for (int j = i + 1; j <= n; ++j) {
        string t = s.substr(i, j - i);
        if (!st.contains(t)) {
          st.insert(t);
          dfs(dfs, j);
          st.erase(t);
        }
      }
    };
    dfs(dfs, 0);
    return ans;
  }
};
```

35 LC 332 | Reconstruct Itinerary (Hard)

Problem: Use every directed flight ticket exactly once to construct an itinerary starting at 'JFK'. If several itineraries exist, return the lexicographically smallest. At least one valid itinerary exists.
Constraints: 1 <= tickets.length <= 300; tickets[i].length == 2; from_i.length == 3; to_i.length == 3; from_i and to_i consist of uppercase English letters.; from_i != to_i

```
class Solution {
public:
  vector<string>
    findItinerary(vector<vector<string>>& tickets)
    {
    sort(tickets.rbegin(), tickets.rend());
    unordered_map<string, vector<string>> g;
    for (const auto& ticket : tickets) {
      g[ticket[0]].push_back(ticket[1]);
    }
    vector<string> ans;
    auto dfs = [&](auto&& dfs, string& f) -> void
      {
      while (!g[f].empty()) {
        string t = g[f].back();
        g[f].pop_back();
        dfs(dfs, t);
      }
      ans.emplace_back(f);
    };
    string f = "JFK";
    dfs(dfs, f);
    ranges::reverse(ans);
    return ans;
  }
};
```

27 LC 2597 | The Number of Beautiful Subsets (Medium)

Problem: Count nonempty index subsets containing no two values with absolute difference k. Equal values at different indices represent separate choices.
Constraints: 1 <= nums.length <= 18; 1 <= nums[i], k <= 1000

```
class Solution {
public:
  int beautifulSubsets(vector<int>& nums, int k) {
    map<int, map<int, int>> groups;
    for (int x : nums) ++groups[x % k][x];
    long long result = 1;
    for (const auto& [remainder, counts] : groups)
      {
      long long skip = 1, take = 0;
      int previous = INT_MIN;
      for (const auto& [value, count] : counts) {
        long long choices = (1LL << count) - 1;
        long long nextTake = choices * (previous
          != INT_MIN && value - previous == k ?
          skip : skip + take);
        skip += take; take = nextTake; previous =
          value;
      }
      result *= skip + take;
    }
    return static_cast<int>(result - 1);
  }
};
```

38 LC 797 | All Paths From Source to Target (Medium)

Problem: Given a directed acyclic graph (DAG) of n nodes labeled from 0 to n - 1, find all possible paths from node 0 to node n - 1 and return them in any order. The graph is given as follows: graph[i] is a list of all nodes you can visit from node i (i.e., there is a directed edge from node i to node graph[i][j]).
Constraints: n == graph.length; 2 <= n <= 15; 0 <= graph[i][j] < n; graph[i][j] != i (i.e., there will be no self-loops).; All the elements of graph[i] are unique.; The input graph is guaranteed to be a DAG.

```
class Solution {
public:
  vector<vector<int>>
    allPathsSourceTarget(vector<vector<int>>&
    graph) {
    vector<vector<int>> result;
    vector<int> path{0};
    auto dfs = [&](auto&& self, int u) -> void {
      if (u == static_cast<int>(graph.size()) - 1)
        { result.push_back(path); return; }
      for (int v : graph[u]) {
        path.push_back(v); self(self, v);
          path.pop_back();
      }
    };
    dfs(dfs, 0);
    return result;
  }
};
```

29 LC 1980 | Find Unique Binary String (Medium)

Problem: Given an array of strings nums containing n unique binary strings each of length n, return a binary string of length n that does not appear in nums. If there are multiple answers, you may return any of them.

Constraints: n == nums.length; 1 <= n <= 16; nums[i].length == n; nums[i] is either '0' or '1'.; All the strings of nums are unique.

```
class Solution {
public:
  string findDifferentBinaryString(vector<string>&
    nums) {
    int n = nums.size();
    string ans(n, '0');
    for (int i = 0; i < n; i++) {
      ans[i] = nums[i][i] == '0' ? '1' : '0';
    }
    return ans;
  }
};
```

30 LC 282 | Expression Add Operators (Hard)

Problem: Given a string num that contains only digits and an integer target, return all possibilities to insert the binary operators '+', '-', and/or '*' between the digits of num so that the resultant expression evaluates to the target value. Note that operands in the returned expressions should not contain leading zeros. Note that a number can contain multiple digits.

Constraints: 1 <= num.length <= 10; num consists of only digits.; -2^31 <= target <= 2^31 - 1

```
class Solution {
public:
  vector<string> addOperators(string num, int
    target) {
    vector<string> result;
    string expression;
    int n = num.size();
    auto dfs = [&](auto&& self, int pos, long long
      value, long long last) -> void {
      if (pos == n) {
        if (value == target)
          result.push_back(expression);
        return;
      }
      long long x = 0;
      auto oldSize = expression.size();
      for (int end = pos; end < n; ++end) {
        if (end > pos && num[pos] == '0') break;
        x = 10 * x + num[end] - '0';
        string token = num.substr(pos, end - pos +
          1);
        if (pos == 0) {
          expression += token;
          self(self, end + 1, x, x);
          expression.resize(oldSize);
        } else {
          for (char op : string_view("+-*")) {
            expression += op;
            expression += token;
            if (op == '+') self(self, end + 1,
              value + x, x);
            if (op == '-') self(self, end + 1,
              value - x, -x);
            if (op == '*') self(self, end + 1,
              value - last + last * x, last * x);
            expression.resize(oldSize);
          }
        }
      }
    };
    dfs(dfs, 0, 0, 0);
    return result;
  }
};
```

31 LC 301 | Remove Invalid Parentheses (Hard)

Problem: Given a string s that contains parentheses and letters, remove the minimum number of invalid parentheses to make the input string valid. Return a list of unique strings that are valid with the minimum number of removals. You may return the answer in any order.

Constraints: 1 <= s.length <= 25; s consists of lowercase English letters and parentheses '(' and ')'.; There will be at most 20 parentheses in s.

```
class Solution {
public:
  vector<string> removeInvalidParentheses(string
    s) {
    unordered_set<string> ans;
    int l = 0, r = 0, n = s.size();
    for (char& c : s) {
      if (c == '(') {
        ++l;
      } else if (c == ')') {
        if (l) {
          --l;
        } else {
          ++r;
        }
      }
    }
    function<void(int, int, int, int, int,
      string)> dfs;
    dfs = [&](int i, int l, int r, int lcnt, int
      rcnt, string t) {
      if (i == n) {
        if (l == 0 && r == 0) {
          ans.insert(t);
        }
        return;
      }
      if (n - i < l + r || lcnt < rcnt) {
        return;
      }
      if (s[i] == '(' && l) {
        dfs(i + 1, l - 1, r, lcnt, rcnt, t);
      }
      if (s[i] == ')' && r) {
        dfs(i + 1, l, r - 1, lcnt, rcnt, t);
      }
      int x = s[i] == '(' ? 1 : 0;
      int y = s[i] == ')' ? 1 : 0;
      dfs(i + 1, l, r, lcnt + x, rcnt + y, t +
        s[i]);
    };
    dfs(0, l, r, 0, 0, "");
    return vector<string>(ans.begin(), ans.end());
  }
};
```

33 LC 212 | Word Search II (Hard)

Problem: Given an m x n board of characters and a list of strings words, return all words on the board. Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Constraints: m == board.length; n == board[i].length; 1 <= m, n <= 12; board[i][j] is a lowercase English letter.; 1 <= words.length <= 3 * 10^4; 1 <= words[i].length <= 10; words[i] consists of lowercase English letters.; All the strings of words are unique.

```
class Trie {
public:
  vector<Trie*> children;
  int ref;
  Trie()
    : children(26, nullptr)
    , ref(-1) {}
  void insert(const string& w, int ref) {
    Trie* node = this;
    for (char c : w) {
      c -= 'a';
      if (!node->children[c]) {
        node->children[c] = new Trie();
      }
      node = node->children[c];
    }
    node->ref = ref;
  }
};
class Solution {
public:
  vector<string> findWords(vector<vector<char>>&
    board, vector<string>& words) {
    Trie* tree = new Trie();
    for (int i = 0; i < words.size(); ++i) {
      tree->insert(words[i], i);
    }
    vector<string> ans;
    int m = board.size(), n = board[0].size();
    function<void(Trie*, int, int)> dfs =
      [&](Trie* node, int i, int j) {
      int idx = board[i][j] - 'a';
      if (!node->children[idx]) {
        return;
      }
      node = node->children[idx];
      if (node->ref != -1) {
        ans.emplace_back(words[node->ref]);
        node->ref = -1;
      }
      int dirs[5] = {-1, 0, 1, 0, -1};
      char c = board[i][j];
      board[i][j] = '#';
      for (int k = 0; k < 4; ++k) {
        int x = i + dirs[k], y = j + dirs[k + 1];
        if (x >= 0 && x < m && y >= 0 && y < n &&
          board[x][y] != '#') {
          dfs(node, x, y);
        }
      }
      board[i][j] = c;
    };
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        dfs(tree, i, j);
      }
    }
    return ans;
  }
};
```

34 LC 126 | Word Ladder II (Hard)

Problem: Return all shortest word sequences from beginWord to endWord, changing exactly one letter per step. Every word after beginWord must occur in wordList; beginWord need not. Return an empty list if no transformation exists.

Constraints: 1 <= beginWord.length <= 5; endWord.length == beginWord.length; 1 <= wordList.length <= 500; wordList[i].length == beginWord.length; beginWord, endWord, and wordList[i] consist of lowercase English letters.; beginWord != endWord; All the words in wordList are unique.; The sum of all shortest transformation sequences does not exceed 10^5.

```
class Solution {
public:
  vector<vector<string>> findLadders(string
    beginWord, string endWord, vector<string>&
    wordList) {
    vector<vector<string>> ans;
    unordered_set<string> words(wordList.begin(),
      wordList.end());
    if (!words.count(endWord)) {
      return ans; }
    words.erase(beginWord);
    unordered_map<string, int> dist{{beginWord,
      0}};
    unordered_map<string, unordered_set<string>>
      prev;
    queue<string> q{{beginWord}};
    bool found = false;
    int step = 0;
    while (!q.empty() && !found) {
      ++step;
      for (int i = q.size(); i > 0; --i) {
        string p = q.front();
        q.pop();
        string t = p;
        for (int j = 0; j < t.size(); ++j) {
          char ch = t[j];
          for (char k = 'a'; k <= 'z'; ++k) {
            t[j] = k;
            if (dist.count(t) && dist[t] == step)
              {
              prev[t].insert(p); }
            if (!words.count(t)) {
              continue; }
            prev[t].insert(p);
            words.erase(t);
            q.push(t);
            dist[t] = step;
            if (t == endWord) {
              found = true; } }
          t[j] = ch; } } }
    function<void(vector<string>&, const string&)>
      dfs = [&](vector<string>& path, const
      string& cur) {
      if (cur == beginWord) {
        ans.emplace_back(path.rbegin(),
          path.rend());
        return; }
      for (const string& precursor : prev[cur]) {
        path.push_back(precursor);
        dfs(path, precursor);
        path.pop_back(); } };
    if (found) {
      vector<string> path{endWord};
      dfs(path, endWord); }
    return ans; } };
```

36 LC 491 | Non-decreasing Subsequences (Medium)

Problem: Given an integer array nums, return all the different possible non-decreasing subsequences of the given array with at least two elements. You may return the answer in any order.
Constraints: 1 <= nums.length <= 15; -100 <= nums[i] <= 100

```
class Solution {
public:
  vector<vector<int>>
    findSubsequences(vector<int>& nums) {
    vector<vector<int>> ans;
    vector<int> t;
    dfs(0, -1000, t, nums, ans);
    return ans;
  }
  void dfs(int u, int last, vector<int>& t,
    vector<int>& nums, vector<vector<int>>& ans) {
    if (u == nums.size()) {
      if (t.size() > 1) ans.push_back(t);
      return;
    }
    if (nums[u] >= last) {
      t.push_back(nums[u]);
      dfs(u + 1, nums[u], t, nums, ans);
      t.pop_back();
    }
    if (nums[u] != last) dfs(u + 1, last, t, nums,
      ans);
  }
};
```

43 LC 1718 | Construct the Lexicographically Largest Valid Sequence (Medium)

Problem: Build a length-(2n-1) sequence: 1 occurs once; each value i=2..n occurs twice, with its two positions differing by i. Return the lexicographically largest valid sequence; a solution exists.
Constraints: 1 <= n <= 20

```
class Solution {
public:
  int n;
  vector<int> cnt, path;
  vector<int> constructDistancedSequence(int _n) {
    n = _n;
    cnt.resize(n * 2, 2);
    path.resize(n * 2);
    cnt[1] = 1;
    dfs(1);
    vector<int> ans;
    for (int i = 1; i < n * 2; ++i)
      ans.push_back(path[i]);
    return ans;
  }
  bool dfs(int u) {
    if (u == n * 2) return 1;
    if (path[u]) return dfs(u + 1);
    for (int i = n; i > 1; --i) {
      if (cnt[i] && u + i < n * 2 && !path[u + i])
        {
        path[u] = path[u + i] = i;
        cnt[i] = 0;
        if (dfs(u + 1)) return 1;
        cnt[i] = 2;
        path[u] = path[u + i] = 0;
      }
    }
    if (cnt[1]) {
      path[u] = 1;
      cnt[1] = 0;
      if (dfs(u + 1)) return 1;
      cnt[1] = 1;
      path[u] = 0;
    }
    return 0;
  }
};
```

40 LC 1219 | Path with Maximum Gold (Medium)

Problem: Start at any positive grid cell and move horizontally/vertically without revisiting cells or entering zero cells. Stop anywhere. Return the maximum total gold collected along one path.
Constraints: m == grid.length; n == grid[i].length; 1 <= m, n <= 15; 0 <= grid[i][j] <= 100; There are at most 25 cells containing gold.

```
class Solution {
public:
  int getMaximumGold(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    function<int(int, int)> dfs = [&](int i, int
      j) {
      if (i < 0 || i >= m || j < 0 || j >= n ||
        !grid[i][j]) {
        return 0;
      }
      int v = grid[i][j];
      grid[i][j] = 0;
      int ans = v + max({dfs(i - 1, j), dfs(i + 1,
        j), dfs(i, j - 1), dfs(i, j + 1)});
      grid[i][j] = v;
      return ans;
    };
    int ans = 0;
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        ans = max(ans, dfs(i, j));
      }
    }
    return ans;
  }
};
```

44 LC 1849 | Splitting a String Into Descending Consecutive Values (Medium)

Problem: Split s into at least two decimal values that strictly decrease by exactly 1 between adjacent parts. Leading zeros are allowed. Use all characters in order; return whether such a split exists.
Constraints: 1 <= s.length <= 20; s only consists of digits.

```
class Solution {
public:
  bool splitString(string s) {
    auto dfs = [&](auto&& dfs, int i, long long x)
      -> bool {
      if (i >= s.size()) {
        return true;
      }
      long long y = 0;
      int r = x < 0 ? s.size() - 1 : s.size();
      for (int j = i; j < r; ++j) {
        y = y * 10 + s[j] - '0';
        if (y > 1e10) {
          break;
        }
        if ((x < 0 || x - y == 1) && dfs(dfs, j +
          1, y)) {
          return true;
        }
      }
      return false;
    };
    return dfs(dfs, 0, -1);
  }
};
```

42 LC 1601 | Maximum Number of Achievable Transfer Requests (Hard)

Problem: Choose the largest subset of building-transfer requests [from,to] such that every building has equal incoming and outgoing transfers. Each request may be accepted at most once, including self-transfers.
Constraints: 1 <= n <= 20; 1 <= requests.length <= 16; requests[i].length == 2; 0 <= from_i, to_i < n

```
class Solution {
public:
  int maximumRequests(int n, vector<vector<int>>&
    requests) {
    int m = requests.size();
    int ans = 0;
    auto check = [&](int mask) -> bool {
      vector<int> cnt = vector<int>(n, 0);
      for (int i = 0; i < m; ++i) {
        if (mask >> i & 1) {
          int f = requests[i][0], t =
            requests[i][1];
          --cnt[f];
          ++cnt[t];
        }
      }
      for (int v : cnt) {
        if (v) {
          return false;
        }
      }
      return true;
    };
    for (int mask = 0; mask < 1 << m; ++mask) {
      int cnt = popcount(static_cast<unsigned
        int>(mask));
      if (ans < cnt && check(mask)) {
        ans = cnt;
      }
    }
    return ans;
  }
};
```

46 LC 967 | Numbers With Same Consecutive Differences (Medium)

Problem: Given two integers n and k, return an array of all the integers of length n where the difference between every two consecutive digits is k. You may return the answer in any order. Note that the integers should not have leading zeros. Integers as 02 and 043 are not allowed.
Constraints: 2 <= n <= 9; 0 <= k <= 9

```
class Solution {
public:
  vector<int> numsSameConsecDiff(int n, int k) {
    vector<int> ans;
    int boundary = pow(10, n - 1);
    auto dfs = [&](auto&& dfs, int x) {
      if (x >= boundary) {
        ans.push_back(x);
        return;
      }
      int last = x % 10;
      if (last + k < 10) {
        dfs(dfs, x * 10 + last + k);
      }
      if (k != 0 && last - k >= 0) {
        dfs(dfs, x * 10 + last - k);
      }
    };
    for (int i = 1; i < 10; ++i) {
      dfs(dfs, i);
    }
    return ans;
  }
};
```

47 LC 306 | Additive Number (Medium)

Problem: Return whether num can be split into at least three decimal integers, each after the first two equaling their sum. A number cannot have a leading zero unless it is zero. All digits must be used in order.
Constraints: 1 <= num.length <= 35; num consists only of digits.

```
class Solution {
  string add(const string& a, const string& b) {
    string sum;
    int i = static_cast<int>(a.size()) - 1, j =
      static_cast<int>(b.size()) - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry) {
      if (i >= 0) carry += a[i--] - '0';
      if (j >= 0) carry += b[j--] - '0';
      sum += static_cast<char>('0' + carry % 10);
      carry /= 10;
    }
    ranges::reverse(sum);
    return sum;
  }
public:
  bool isAdditiveNumber(string num) {
    int n = num.size();
    for (int first = 1; first < n - 1; ++first) {
      if (first > 1 && num[0] == '0') break;
      for (int second = first + 1; second < n;
        ++second) {
        if (second > first + 1 && num[first] ==
          '0') break;
        string a = num.substr(0, first), b =
          num.substr(first, second - first);
        int pos = second;
        while (pos < n) {
          string next = add(a, b);
          if (num.compare(pos, next.size(), next)
            != 0) break;
          pos += next.size();
          a = move(b); b = move(next);
        }
        if (pos == n) return true;
      }
    }
    return false;
  }
};
```

48 LC 60 | Permutation Sequence (Hard)

Problem: The set [1, 2, 3, ..., n] contains a total of n! unique permutations. By listing and labeling all of the permutations in order, we get the following sequence for n = 3: - "123" - "132" - "213" - "231" - "312" - "321" Given n and k, return the kth permutation sequence.
Constraints: 1 <= n <= 9; 1 <= k <= n!

```
class Solution {
public:
  string getPermutation(int n, int k) {
    string ans;
    bitset<10> vis;
    for (int i = 0; i < n; ++i) {
      int fact = 1;
      for (int j = 1; j < n - i; ++j) fact *= j;
      for (int j = 1; j <= n; ++j) {
        if (vis[j]) continue;
        if (k > fact)
          k -= fact;
        else {
          ans += to_string(j);
          vis[j] = 1;
          break;
        }
      }
    }
    return ans;
  }
};
```